Draw It or Lose It

CS 230 Project Software Design Template

Version 1.2

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2

Document Revision History 2

Executive Summary 3

Requirements 3

Design Constraints 3

System Architecture View 3

Domain Model 3

Evaluation 4

Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	03/22/2026	Erica Kinch	Completed Executive Summary, Design Constraints, and Domain Model
1.1	03/31/2026	Erica Kinch	Completed Evaluation and Recommendations sections; reformatted Evaluation into two tables for readability.
1.2	04/19/2026	Erica Kinch	Final revision pass

Executive Summary

The Gaming Room has asked Creative Technology Solutions (CTS) to help expand its existing Android game, *Draw It or Lose It*, into a web-based application that supports multiple operating systems. Recognizing the importance of this project, we aim to select a software architecture that best supports multiplayer gameplay, enforces unique names for games and teams, and centralizes application logic and data management, ensuring the team feels confident in the plan.

A web-based client-server design is the strongest solution because it improves platform accessibility, supports future growth, and keeps core processing centralized on the server. Using Java object-oriented design principles, the application can be organized around shared entities and managed through a single GameService instance. The singleton pattern ensures that one service instance controls in-memory game data, while iterator-based searches help enforce unique names for games, teams, and players. Inheritance through the Entity base class further improves maintainability by reducing duplication across Game, Team, and Player. This design provides The Gaming Room with a scalable, maintainable, and platform-independent foundation for launching and expanding the application, setting the stage for future success.

Requirements

The Gaming Room needs *Draw It or Lose It* converted from an Android-only game into a web-based application that supports multiple platforms. The system must allow multiplayer participation, enforce unique names for games and teams, and support centralized data management. Because the project is still in the design phase, the client needs a software architecture recommendation before selecting hardware or deployment infrastructure.

The system must also be scalable for future growth and maintainable for long-term development. These requirements establish the foundation for selecting an appropriate architecture and platform strategy.

Design Constraints

The design of *Draw It or Lose It* must address several technical and operational constraints as The Gaming Room expands the application from a single-platform mobile game into a web-based system used across multiple environments. These constraints shape the recommended architecture, platform strategy, and object-oriented design decisions.

Cross-platform compatibility:

Because the game must move beyond Android and become web-based, the system must function across different operating systems and web browsers. This limits the usefulness of device-specific solutions and favors a browser-based system with centralized server-side logic. The design should emphasize platform independence rather than device-specific implementation.

Distributed environment:

A web-based game operates in a distributed environment where users connect across networks rather than from a single local device. This introduces constraints related to latency, connection reliability, synchronization, and centralized coordination of game activity. These factors affect response time, session coordination, and overall user experience.

Scalability:

Because the application supports multiplayer gameplay, the system may need to accommodate increasing numbers of users, teams, and active sessions over time. The architecture should scale without requiring major redesign, making flexibility and efficient resource management important design considerations.

Data consistency and uniqueness:

The client requires unique names for games and teams, and the architecture must incorporate centralized validation mechanisms to prevent duplicates and ensure data integrity across distributed sessions. Detailing security measures will reassure stakeholders about data consistency and system robustness.

Single service control:

The design requires one shared service layer to manage in-memory game data and validation. This constrains how the service layer is structured and supports the use of the singleton pattern in GameService. While this improves consistency, it also requires careful organization to keep the service layer maintainable.

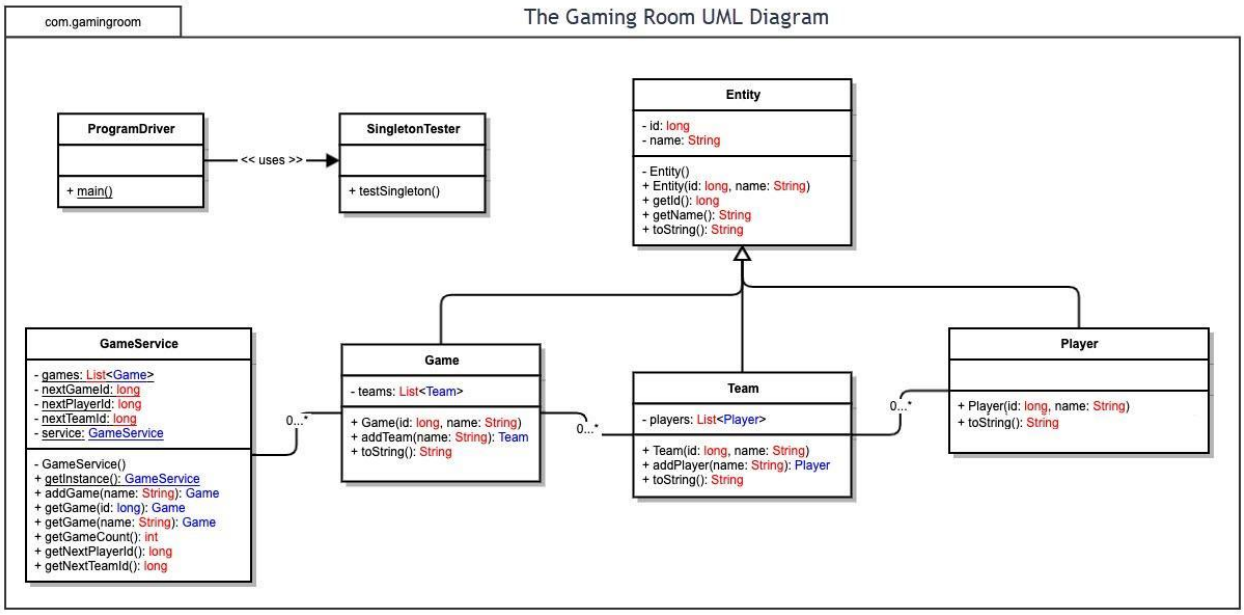
Maintainability:

The application is being expanded for long-term use and future growth, so the design should reduce duplication and make updates easier to implement. Reusable class structures, clear relationships, and consistent naming conventions are important design constraints. This is one reason inheritance through the Entity base class is beneficial.

System Architecture View

A detailed system architecture view is not required for Project One. At a high level, the proposed solution is a web-based client-server application in which users access the game through web browsers on client devices, while application logic and data management remain centralized on the server. This approach ensures a reliable, scalable system that can adapt to future needs, giving the team confidence in its robustness and flexibility.

Domain Model



The domain model organizes the application around the core gameplay objects: Game, Team, and Player. Each class extends the Entity base class, which provides shared attributes such as id and name. This use of inheritance centralizes common features in a parent class rather than duplicating them across multiple child classes, making the design more consistent and maintainable.

The relationships among these classes reflect the structure of the game itself. A Game contains one or more Team objects, and each Team contains multiple Player objects. These associations show how the system is logically organized. Encapsulation is also present because each class manages its own data and behavior through constructors and methods such as addTeam() and addPlayer() rather than exposing uncontrolled outside access.

In addition, GameService functions as the central service layer for managing game data in memory. It uses the singleton pattern so that only one shared instance controls application state. Iterator-based searches are then used to enforce unique names for games, teams, and players. Together, these object-oriented design choices create a maintainable structure that supports the client's requirements and future growth.

Evaluation

Platform Evaluation

Platform	Server Side	Client Side
Mac	macOS can host a web-based application and supports Java, web servers, and browser-based delivery. Its strengths are stability and support for the Apple ecosystem, but it is less common in enterprise hosting and requires more expensive Apple hardware. In addition to higher hardware costs, it does not offer the same cost advantages as Linux for large-scale server deployment. For The Gaming Room, Mac is technically possible, but not the most cost-effective or scalable server option.	Mac clients are best supported through a browser-based interface rather than a separate native application. This avoids the cost of platform-specific development, though the team still needs to test across Safari, Chrome, and Firefox for compatibility.
Linux	Linux is the strongest server-side option because it is stable, scalable, secure, and widely used for web hosting and distributed systems. It works well with Java application servers, databases, and cloud platforms, and many distributions have minimal or no operating system licensing costs. This gives The Gaming Room a clear cost advantage compared with Windows Server. Its main weakness is that it may require more technical expertise to manage, but it offers the best overall	Linux clients can use the same browser-based interface, keeping development costs low. The main consideration is testing across different browsers and desktop environments, since Linux user setups are less standardized.

	balance of cost, performance, and flexibility.	
Windows	Windows Server can host web-based applications and supports Java, web servers, and enterprise deployment tools. Its main strengths are familiarity, commercial support, and integration with Microsoft tools. However, Windows Server has higher operating system licensing costs than Linux and typically uses more resources, which can increase overall infrastructure costs. This makes it a viable but more costly server option for The Gaming Room.	Windows clients are also well served by a browser-based interface. Because Windows will likely represent a large share of users, compatibility testing across Edge, Chrome, and Firefox is especially important.
Mobile Devices	Mobile devices are not suitable for hosting a distributed web-based application. They are designed to function as clients, not centralized servers, and they lack the scalability, uptime, and resource control needed to support thousands of players. Because they are not realistic server-hosting platforms for this project, server operating system licensing costs are not applicable in this category.	Mobile support adds complexity because the interface must work across Android and iOS, on smaller screens, with touch input, and in changing orientations. A responsive web design keeps development more efficient than separate native apps, but it increases the need for testing, front-end optimization, and usability validation.

Development Tools

Most development for this project would rely on a shared cross-platform toolset, including Java for the back end, HTML/CSS/JavaScript for the front end, Git for version control, and IDEs such as IntelliJ IDEA, Eclipse, and VS Code. Platform differences mainly affect testing requirements, hardware access, and possible licensing costs.

Platform	**Tools**
Mac	Mac development supports the same Java and web toolchain used on other platforms, but reliable Safari testing requires access to Mac hardware. That raises equipment costs compared with platform-agnostic browser testing.
Linux	Linux supports the same core development stack and is especially well-suited to server-side development and deployment. Many related tools are open source, which helps keep licensing costs low, though the team may need stronger Linux administration skills.
Windows	Windows supports the full development stack and is practical because many developers already use it as a workstation environment. The main tradeoff is that some enterprise tools may introduce additional licensing costs.
Mobile Devices	Mobile browser support still relies on the same core web technologies rather than separate native SDKs. The main impact is increased testing across Android and iOS devices, screen sizes, and interaction patterns, though this remains less costly than full native mobile development.

Recommendations

1. **Operating Platform**:

   The most appropriate operating platform for The Gaming Room is a Linux-based server environment with a browser-based client interface. This approach best supports the company's goal of expanding *Draw It or Lose It* from an Android-only application into a web-based game accessible across Windows, macOS, Linux desktops, and mobile devices. Linux is the strongest server-side choice because it is widely used for web hosting, scales well, and typically has lower licensing costs than Windows Server or Apple-based alternatives. A browser-based front end also eliminates the need to maintain separate native applications for each client platform, reducing long-term development and maintenance costs.

2. **Operating Systems Architectures**:

   The recommended architecture is a Linux-hosted client-server web application. In this model, the server handles core application logic, validation, session management, and centralized data access, while the client manages presentation and user interaction through a web browser. This separation improves portability because users can access the game from multiple operating systems without requiring platform-specific implementations of the core logic. Linux is especially effective in this model because it supports Java-based application servers, web services, databases, and cloud deployment environments well.

3. **Storage Management**:

   An appropriate storage solution for this platform is a relational database management system such as MySQL or PostgreSQL running alongside the Linux server environment. A relational database is a good fit because the application relies on structured data, including games, teams, players, and user sessions, and must enforce consistency and uniqueness rules across that data. This is especially important because The Gaming Room requires unique names for games and teams. Compared with file-based storage, a relational database better supports concurrent access, data integrity, scalability, and long-term maintainability.

4. **Memory Management**:

   Linux uses memory management techniques such as virtual memory, process isolation, and efficient resource allocation, which make it well-suited for hosting a web-based multiplayer

application like *Draw It or Lose It*. These features allow the server to manage multiple active processes and user sessions without one process interfering with another. Java also provides a managed runtime environment and garbage collection, reducing direct memory errors and making memory usage more predictable. Together, Linux and Java provide a stable foundation for handling the service layer, active sessions, and game state as usage increases.

5. **Distributed Systems and Networks**:

   Cross-platform communication should be handled through a distributed web application model in which client devices connect to a centralized server using HTTP and HTTPS. In this design, Windows, macOS, Linux, Android, and iOS devices all access the same back-end application through web browsers. This keeps business logic and data centralized while simplifying deployment. Because distributed systems depend on network reliability and server uptime, the design should also include stable session handling, monitoring, and recovery mechanisms to minimize disruption caused by outages, latency, or dropped connections.

6. **Security**:

   Security should be treated as a core design requirement rather than an afterthought. To protect user information across platforms, the recommended Linux-based environment should use secure communication, strong authentication, careful authorization, and protected data storage. All communication between clients and the server should occur over HTTPS so data is encrypted in transit. User credentials should be stored with password hashing rather than in plain text, and authorization rules should be enforced not only at the login and endpoint level but also within the application's service logic. This layered approach is especially important in a distributed application, where protecting only the outer layer is insufficient. Linux also provides strong security features, including access controls, permission management, and process isolation. Combined with secure application design, these protections give The Gaming Room a stronger foundation for protecting user information across desktop and mobile platforms.